SevaFlow

Website Product Requirements Document

Prototype scope | Version 1.0 | 23 October 2026

Status: Proposed design. No real clinical, government or ABDM deployment is asserted.

Website-specific experience

Deliver responsive browser experiences for patient self-service, reception, doctors, administrators and public displays. Provide navigation appropriate to each role; never expose administrative functionality through patient navigation. Government demonstration is a separate restricted view. A shared login page redirects staff to authorized workspaces; patient access uses a prototype authentication flow with clearly labelled simulated OTP if no provider exists.

Website screen inventory

Public landing and QR entry; registration and department selection; patient token tracker; notification and caregiver consent; receptionist registration and searchable queue; doctor workspace with availability; public fullscreen token board; hospital operations dashboard; alert response panel; feedback review; audit viewer; configuration; district overview and facility trends; connection/error states. Each screen requires empty, loading, error and success states.

Website interaction requirements

Receptionists can complete keyboard-first registration and print a minimal token. Doctors call, start and complete visits without re-entering patient data. Administrators filter metrics by facility, department and date with definitions visible. Public displays use a dedicated read-only credential or scoped session. QR entry resolves a facility reference only, not embedded patient information. Browser audio restrictions are handled through an explicit enable-announcements control.

Website architecture and deployment

Suggested architecture: responsive React/Next.js frontend, a shared REST backend, PostgreSQL and a queue-update channel. This is a recommendation, not a fixed stack requirement. Keep public display endpoints read-only, apply server-side role checks and separate analytics queries from high-contention queue transactions. Deploy with HTTPS, environment-based secrets, seeded synthetic data, health checks and repeatable migrations. Test current desktop and mobile browsers.

Website acceptance scenarios

Print a token without exposing contact details; receptionist and mobile registration share the same department queue; keyboard-only staff can complete the main flow; two doctor workspaces cannot call the same visit; fullscreen board recovers after reconnect; unauthorized district users cannot access patient APIs; exported reports, if implemented, follow role scope and suppress identifying data. Export actions require explicit user initiation and audit events.

1. Product vision and scope

SevaFlow is an inclusive hospital token and patient-flow platform. It connects patients, caregivers, receptionists, doctors, hospital administrators and a privacy-preserving government demonstration dashboard. The complete proposed scope includes all features discussed: core queues, assisted access, accessibility, controlled priority, waiting estimates, notifications, caregiver access, operations analytics, district insights, alerts, feedback, audit

trails, ABDM integration readiness and intermittent-connectivity handling. This PRD describes a prototype, not a certified clinical or government system.

2. Goals and success measures

Goals: make the next action clear to patients; reduce manual queue coordination; support access without a smartphone; make staff actions traceable; expose operational bottlenecks without exposing patient identities. Measure registration completion time, median and 90th-percentile wait, estimate error, successful assisted registrations, duplicate-token failures, notification delivery outcomes and audit completeness. Establish baseline and target values during testing. Do not claim clinical outcomes or percentage improvements without measured evidence.

3. Roles and authorization

Patient: own registration, tokens, estimates and permissions. Caregiver: explicitly authorized queue information only. Receptionist: registration, department selection and authorized queue actions. Doctor: assigned queue, consultation start and completion. Hospital administrator: configuration, operations, audit review and feedback. Government analyst: aggregated synthetic multi-facility data in the prototype. Enforce hospital boundaries and role permissions on the server, not only in the interface. Public displays expose token and destination only.

4. Registration and inclusive access

Support app registration, browser QR registration, receptionist-assisted entry and printable tokens feeding one queue. Minimum fields: display name, department and required operational identifiers; contact and caregiver details are optional unless a feature needs them. Validate required fields, department availability and submission format. Use request idempotency so retries do not create duplicate registrations. Offer manual entry without requiring ABHA. Token pickup and check-in rules must be explicit. Clarify with organizers whether the printed Recipient module means receptionist; this PRD uses receptionist as a working assumption.

5. Queue engine and state model

Use a transactionally unique token scoped to facility, service date and department, with a separate immutable internal ID. Required states: Waiting, In Progress and Completed. Additional events include Called, Skipped, Cancelled and No-show; store them explicitly without confusing them with the required states. Normal order is first eligible checked-in patient first, subject to documented hospital priority rules. Call-next must be atomic to prevent two staff members selecting the same patient. Record call, arrival, start and completion times separately. Allow authorized recall and correction with reasons. Department transfers preserve history and issue a linked destination token.

6. Priority and clinical safety

Only authorized staff assign priority categories using hospital-approved rules. Every override records actor, timestamp, reason and previous ordering information. Public views must not reveal the medical reason. Show emergency guidance directing patients to clinical staff immediately. No automated diagnosis, autonomous symptom-based triage or emergency eligibility decisions. Configure missed-turn and priority policies before demonstration; provide human review for unusual actions.

7. Live updates and waiting estimates

Publish queue changes to connected interfaces using WebSocket or server-sent events, with automatic polling fallback. Show last-updated time and stale-state indicators. Estimate a waiting-time range from recent completed consultation durations, eligible patients ahead, active doctors and doctor availability. Label estimates as approximate; fall back to a stated configurable average when observations are insufficient. Recompute on queue changes, priority changes and service pauses. Measure estimate error on synthetic test sessions; do not imply

validated prediction accuracy.

8. Accessibility, language and announcements

Provide English, Hindi and one selected regional language using reviewed translations. Support scalable text, keyboard navigation on web, screen-reader labels, adequate contrast, descriptive status text and non-colour cues. Target WCAG 2.2 AA in design and test relevant flows; do not claim certification. Public visual and spoken announcements identify token and room only. Prototype announcements use reviewed templates or recordings rather than unverified medical translation.

9. Notifications and caregiver permissions

Offer explicit opt-in for approaching-turn, called, delay and cancellation notices. Support in-app notices and a provider adapter for push or SMS; show simulated delivery when no provider is connected. Store notification event, destination reference, delivery state and retry status without leaking medical details. A patient can authorize and revoke caregiver access to queue updates; caregiver permission does not grant clinical-record access. Never promise delivery or guarantee a turn based solely on a notification. Show arrival, check-in and missed-turn rules.

10. Operations and government benefits

Hospital dashboard includes waiting count, active doctors, median and P90 wait, registration wait separately from consultation wait, arrivals by hour, completed visits, missed turns and threshold breaches. Acknowledge rule-based crowding alerts and record operational responses. District demonstration aggregates three fictional hospitals with service availability, queue load, trends and data freshness. Suppress small groups under a configurable privacy threshold and restrict filters that could re-identify patients. No identifiable patient drill-down for government roles. These are planning indicators, not automatic staff rankings or funding decisions.

11. Feedback and audit

Collect optional short feedback and separate queue-service complaints from clinical complaints. Provide a review status and authorized resolution workflow; this is not an emergency channel. Append audit records for registration, calling, status changes, access changes, priority overrides and configuration edits. Restrict audit access and prevent ordinary users from editing history. A prototype audit log is traceability, not a guarantee against tampering or corruption.

12. ABDM readiness and integration boundaries

Provide a clearly labelled mock consented Scan and Share registration adapter. Manual registration remains available. Use fictional identifiers; never request a real ABHA during the demo. Live ABDM integration requires the relevant official onboarding, sandbox validation and permissions; do not label the prototype ABDM integrated until verified. Health-record exchange is outside the implemented queue scope. Having an identifier alone must not grant medical-record access.

13. Privacy and security requirements

Use synthetic patients only. Minimize collection, explain purposes, separate optional notification permissions and provide revocation. Enforce role-based access, facility isolation, encrypted transport, protected credentials, session expiry, input validation and rate limits. Do not place tokens carrying personal data in public QR URLs. Define retention and deletion per record category before any production pilot; retain only what is justified. No public diagnosis, phone number or full-name display. Production requires security review, applicable legal assessment and institutional approvals; consent alone is not compliance.

14. Shared data model and services

Entities: Facility, Department, Room, User, RoleAssignment, PatientProfile, Visit, Token, QueueEvent, DoctorAvailability, PriorityDecision, CaregiverPermission, Notification, Feedback, Alert, OperationalResponse and AuditEvent. Store timestamps in UTC and render facility-local time. Suggested API capabilities: register visit, get own token, list authorized queues, atomic call-next, start/complete consultation, update availability, set priority with reason, authorize/revoke caregiver, submit feedback and retrieve role-scoped aggregates. Version APIs and standardize errors. Use one shared backend for both products.

15. Reliability and connectivity

Connected queue updates should appear within a prototype target of 3 seconds under the declared test load; fallback refresh every 10 seconds. Show connection state immediately when detected. Cache only minimum permitted data. Offline registration creates a local pending request, not an authoritative token; issue the token only after server acceptance. Reconcile using idempotency keys and surface conflicts. Show stale estimates and disable unsafe offline call-next actions. Test simultaneous registrations, repeated taps, network drops and recovery.

16. Acceptance tests and evidence

Required acceptance: two concurrent registrations never receive the same scoped token; simultaneous call-next requests select distinct patients; all core statuses propagate; unauthorized users cannot read another patient or hospital; caregiver revocation removes future access; priority changes include reasons and audit entries; public displays omit personal details; offline retries create one visit; government views contain synthetic aggregates only; language switching preserves task state. Record pass/fail, test data, latency and screenshots. Accessibility and security checks must include negative cases.

17. Delivery plan and scope control

Phase 1: working registration, token engine, roles and core web/mobile flow. Phase 2: live updates, estimates, accessibility, caregiver permissions and notices. Phase 3: priority audits, feedback, operations alerts and district simulation. Phase 4: mock ABDM adapter, connectivity handling, concurrency/security tests and presentation rehearsal. All features remain in the target scope, but distinguish implemented, simulated and planned capabilities in the final demonstration. Billing, prescriptions, medical records, autonomous triage, real district deployment and verified production interoperability are not included.

18. Demonstration and release gate

Demonstrate assisted and app registration, a bilingual announcement, a consented caregiver view, a staff priority override, a doctor delay, updated estimates, a crowding alert and response, completion, feedback and aggregate district reporting. Use three fictional facilities and sufficient synthetic visits to populate metrics. Release the demo only when core registration-to-completion works, access checks pass, mock services are labelled, no real patient data is present and measured results are separated from projected benefits.

19. Evidence and assumptions

Source of core requirements: user-provided hackathon page, Smart Token and Queue Management & Hospital Staff Dashboard. Product additions are proposed design choices, not organizer requirements. Background references: WHO Digital Health (https://www.who.int/health-topics/digital-health); ABDM Scan and Share FAQs (https://abdm.gov.in/scan-share/faqs); ABDM Health Data Management Policy (https://abdm.gov.in/strapicms/uploads/health_management_policy_bac9429a79.pdf). These references provide policy and ecosystem context, not a certification or claim that the prototype complies. Open decisions: team size, time limit, selected language, deployment stack, institutional queue policy, notification provider and meaning of Recipient module.